

Практическая работа № 1

Поддержка и тестирование программных модулей
на тему: Создание и запуск модульных тестов
для управляемого кода

Специальность «Информационные системы и программирование»

Руководитель

_____ / Д.Б. Берестов/

Оценка _____

« ____ » _____ 2026 г.

Исполнитель

_____ / Н.А. Родыгин/

Группа ИПо-32

« ____ » _____ 2026 г.

Пермь, 2026

Оглавление

- 1. Цель и постановка задачи . . . 3
- 2. Среда и структура решения . . . 4
- 3. Обнаружение и исправление ошибки . . . 5
- 4. Проверка исключений и рефакторинг . . . 6
- 5. Результаты проверки . . . 7
- 6. Запуск проекта и выводы . . . 8
- 7. Используемые материалы . . . 9

Нажмите на название раздела для перехода. В PDF также доступны закладки.

1. Цель и постановка задачи

Цель работы - освоить создание модульных тестов C# с помощью MSTest, обнаружение дефектов по результатам тестирования и повторную проверку после исправления кода.

Работа выполнена по страницам 158–170 руководства «Средства тестирования Visual Studio-2022». Объект тестирования - метод Debit класса BankAccount, моделирующий снятие средств с банковского счёта.

Проверяются три сценария: допустимая сумма списывается с баланса; отрицательная сумма вызывает ArgumentException; сумма больше остатка также вызывает ArgumentException.

Каждый тест использует схему Arrange–Act–Assert: подготовка исходных данных, вызов метода, проверка результата. Для сравнения значений double задан допуск 0,001, как в руководстве.

Учебная модель сохранена: для баланса используется double. Это демонстрационный пример модульного тестирования, а не реализация банковской системы.

2. Среда и структура решения

Решение Bank.sln содержит консольный проект Bank и тестовый проект BankTests. Тестовый проект ссылается на основной через ProjectReference.

```
Bank.sln
Bank/
  Bank.csproj
  BankAccount.cs
BankTests/
  BankTests.csproj
  BankAccountTests.cs
evidence/
  *.trx
  *.log
README.md
report.pdf
```

Целевая платформа - .NET 8.0. Версии пакетов зафиксированы: Microsoft.NET.Test.Sdk 17.11.1, MSTest.TestAdapter 3.6.4 и MSTest.TestFramework 3.6.4. Для открытия решения требуется Visual Studio 2022 с поддержкой .NET 8.

Сборка и тестирование выполнены через .NET CLI. Работа в графическом интерфейсе Visual Studio и снимки её Обозревателя тестов не имитировались. Инструкция повторного запуска в Visual Studio приведена в разделе 6.

3. Обнаружение и исправление ошибки

Первый тест `Debit_WithValidAmount_UpdatesBalance` создаёт счёт с балансом 11,99, списывает 4,55 и сравнивает остаток с ожидаемым значением 7,44.

```
double beginningBalance = 11.99;
double debitAmount = 4.55;
double expected = 7.44;
account.Debit(debitAmount);
Assert.AreEqual(expected, account.Balance, 0.001);
```

Для начального запуска воспроизведена ошибочная строка руководства:

```
m_balance += amount;
```

При такой реализации получается 16,54 вместо 7,44. Тест должен завершиться ошибкой, поскольку баланс увеличивается вместо уменьшения. Фактический результат запуска сохранён в `evidence/01-bug.trx` и соответствующем журнале.

Исправление меняет сложение на вычитание:

```
m_balance -= amount;
```

Повторный запуск того же теста сохранён в `evidence/02-fixed.trx`. В передаваемом исходном коде находится исправленный вариант.

4. Проверка исключений и рефакторинг

Для отрицательной суммы используется -100,00 при балансе 11,99; для превышения остатка - сумма 20,00 при том же балансе. Сначала тип исключения проверяется с помощью `Assert.ThrowsException<ArgumentOutOfRangeException>`. Результат этого этапа сохранён в `evidence/03-exceptions.trx`.

После рефакторинга класс содержит две константы сообщений: `DebitAmountLessThanZeroMessage` и `DebitAmountExceedsBalanceMessage`. Конструктор исключения принимает имя параметра, недопустимое значение и сообщение.

```
throw new ArgumentOutOfRangeException(  
    "amount", amount,  
    DebitAmountExceedsBalanceMessage);
```

Итоговые тесты используют `try/catch` и `StringAssert.Contains` по образцу руководства. После успешных проверок выполняется `return`. За блоком `catch` расположен `Assert.Fail`: если исключение не возникнет, тест завершится ошибкой.

```
catch (ArgumentOutOfRangeException e)  
{  
    StringAssert.Contains(e.Message,  
        BankAccount.DebitAmountExceedsBalanceMessage);  
    // Additional checks: parameter, value, balance.  
    return;  
}  
Assert.Fail("The expected exception was not thrown.");
```

Дополнительно проверяются `ParamName`, `ActualValue` и неизменность баланса после отклонённой операции. Эти проверки уточняют причину отказа и выявляют побочные изменения состояния.

5. Результаты проверки

Исходный дефект: неверный знак операции

Всего: 1; успешно: 0; с ошибкой: 1. Файл: evidence/01-bug.trx.

После исправления знака операции

Всего: 1; успешно: 1; с ошибкой: 0. Файл: evidence/02-fixed.trx.

Проверка типа исключений через ThrowsException

Всего: 2; успешно: 2; с ошибкой: 0. Файл: evidence/03-exceptions.trx.

Итоговый набор после рефакторинга

Всего: 3; успешно: 3; с ошибкой: 0. Файл: evidence/04-final.trx.

Первый отрицательный результат ожидаем: он подтверждает обнаружение намеренно внесённого дефекта. После исправления исходного кода итоговый набор должен проходить полностью.

Контрольный запуск консольного приложения:

```
Current balance is $6,54
```

Последовательность операций: $11,99 + 5,77 - 11,22 = 6,54$. Вывод форматируется до двух знаков после запятой; разделитель зависит от культуры среды.

TRX-файлы содержат реальные результаты выполнения MSTest. Журналы запусков приложены в каталоге evidence. Сведения о SDK сохранены в sdk-info.txt.

6. Запуск проекта и выводы

В Visual Studio 2022. Открыть Bank.sln, дождаться восстановления NuGet-пакетов, выполнить «Сборка → Построить решение», открыть «Тест → Обозреватель тестов» и выбрать «Запустить все». Для консольного запуска выбрать Bank запускаемым проектом.

Из командной строки. В каталоге решения выполнить:

```
dotnet restore Bank.sln
dotnet build Bank.sln --no-restore
dotnet test Bank.sln --no-build
dotnet run --project Bank/Bank.csproj
```

Полный исходный код находится в Bank/BankAccount.cs и BankTests/BankAccountTests.cs. Сопроводительная записка включена в комплект под именем report.pdf; оглавление содержит внутренние ссылки.

Вывод. Разработан набор из трёх модульных тестов для метода Debit. Первый тест обнаруживает ошибку арифметической операции. Проверки исключений контролируют недопустимые входные данные. Рефакторинг делает сообщения информативными, а Assert.Fail и return исключают ложноположительный результат при отсутствии исключения.

Подготовленный проект демонстрирует цикл «ошибка → тест → исправление → повторный тест» и готов к повторному запуску средствами MSTest.

7. Использованные материалы

1. «Средства тестирования Visual Studio-2022». Предоставленный PDF, страницы 158–170. Раздел «Пошаговое руководство. Создание и запуск модульных тестов для управляемого кода», дата статьи 19.06.2023.

2. Microsoft. Дистрибутив .NET 8:
dotnet.microsoft.com/en-us/download/dotnet/8.0.

3. NuGet. MSTest.TestFramework 3.6.4:
nuget.org/packages/MSTest.TestFramework/3.6.4.

Дата подготовки: 09.10.2026. Исходное руководство использовано как источник задания; полный PDF руководства не включён в репозиторий.